

```
(s-capitalized? "Foo bar")
t
```

You can verify that the input string has only numerals using the *s-numeric?* predicate function. If it contains only numbers, it returns true *t*. Otherwise, it returns *nil*. This is illustrated in the following examples:

```
(s-numeric? "10")
t
```

```
(s-numeric? "A10")
nil
```

Miscellaneous functions

There are a few miscellaneous functions in the *s.el* library. The *s-replace* function takes three arguments: an old string, a new string, and an input string. It replaces the occurrence of the old string with the new string in the input string. For example:

```
(s-replace "main" "input" "src/main.js")
"src/input.js"
```

You can make all the letters in a string lowercase using the *s-downcase* function, and make all the letters uppercase using the *s-upcase* function. A couple of examples are given below:

```
(s-downcase "FOO")
"foo"
```

```
(s-upcase "foo")
"FOO"
```

The *s-capitalize* function is used to capitalise the first letter of the input string as shown below:

```
(s-capitalize "pictures")
"Pictures"
```

You can reverse a string using the *s-reverse* function. For example:

```
(s-reverse "654321")
"123456"
```

Word functions

The *s.el* library also provides a number of functions that operate on words. The *s-split-words* function can break the words that are separated by hyphens, the underscore or those with camel case. For example:

```
(s-split-words "i-want-2-do-project.tell-me-wat-2-do")
```

```
("i" "want" "2" "do" "project" "tell" "me" "wat" "2" "do")
```

You can make an input string lower camel case using the *s-lower-camel-case* function, and upper camel case using the *s-upper-camel-case* function, respectively. A couple of examples are shown below:

```
(s-lower-camel-case "TheMain")
"theMain"
```

```
(s-upper-camel-case "foobar")
"Foobar"
```

If you would like to use snake case, as in Ruby variable names, you can use the *s-snake-case* function. For example:

```
(s-snake-case "CRUD API")
"crud_api"
```

The source code definition of *s-snake-case* is given below for reference:

```
(defun s-snake-case (s)
  "Convert S to snake_case."

  (declare (side-effect-free t))

  (s-join "_" (mapcar 'downcase (s-split-words s))))
```

It splits the words, makes them lowercase, and joins them using the "_" character.

The *s-capitalized-words* function can convert an input string to capitalised words as illustrated below:

```
(s-capitalized-words "monty python")
"Monty python"
```

The *s-word-initials* function returns the initials of each word in an input string as shown below:

```
(s-word-initials "Get Things Done")
"GTD"
```

You are encouraged to refer to the README file at <https://github.com/magnars/s.el/blob/master/README.md> for more examples and use cases. 

 By: Shakthi Kannan

The author is a free software enthusiast and blogs at shakthimaan.com.